

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Maclin, R., and D. Optiz. "Popular Ensemble Methods: An Empirical Study." *Journal of Artificial Intelligence Research*, vol. 11, pp. 169–198, 1999, <https://arxiv.org/abs/1106.0257>.

- **Required pages:** pp. 2–4
- **Estimated reading time:** 20 minutes

MACHINE LEARNING IN FINANCE

MODULE 4 | LESSON 1

ENSEMBLE LEARNING

Reading Time	18 minutes
Prior Knowledge	Naive Bayes, Support Vector Machines, Decision Tree, Train and Test set.
Keywords	Blending, Bagging, Stacking, Validation.

*Thus far, we have covered a variety of machine learning techniques specifically classifiers. Oftentimes, we look for various opinions and then use the majority to guide our decision in the end. Think of ratings/feedback on a product advertised online. Ensemble learning works in a similar way. Decisions from various classifiers are combined and a majority vote or weighted average is taken as the final decision. This has some benefits such as improving the accuracy, reducing the generalization error of the individual classifiers (Opitz and Maclin 169) and reducing the variance error (Ali and Pazzani 173) without compromising bias. We've touched on ensemble learning in Module 3 when we mentioned boosting, the idea of combining weak learners to make a single strong one. We look at an ensemble method, specifically **Bagging** in the next section.*

```
In [1]: import pandas as pd
```

1. Bagging

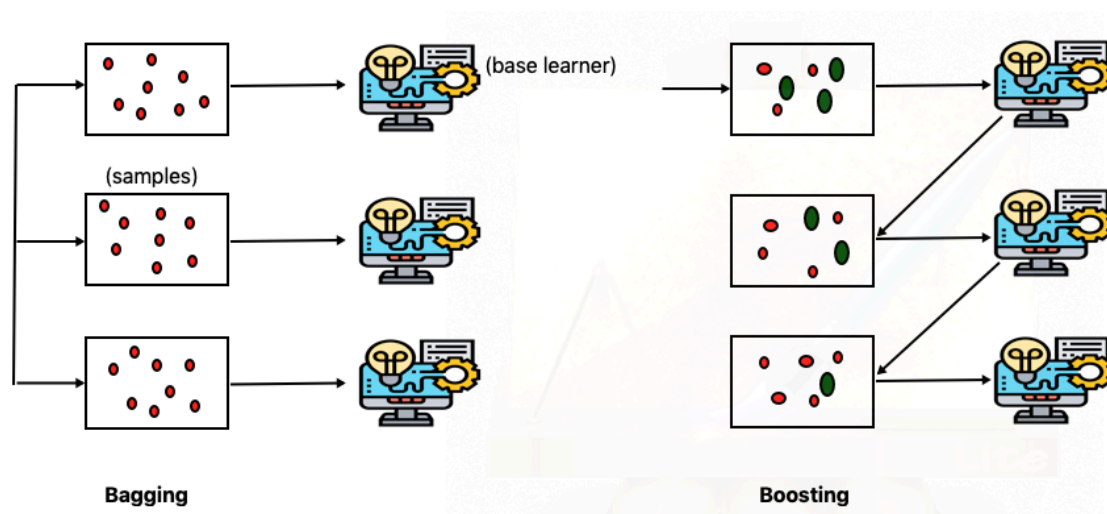
Bagging or bootstrap aggregation is a way to reduce variance within a noisy dataset. In this process, weak classifiers are trained independently on a sample instead of the complete training set. These samples are not necessarily smaller than the training set; however, the sample size is often the same size as the training set to ensure each classifier has sufficient data to train. The sample is drawn with replacement; hence, observations can be exposed to more than one classifier and samples would have observations more than once. The weak classifiers do not necessarily have the same predictors or features. Each weak classifier will likely have different ways of predicting target classes not only on different features but also different splits on the same feature since the samples are different. This diversity among the weak learners is caused by an *inducer*. A stable inducer results in similar weak learners, which produce similar predictions that are unlikely to improve the performance of the model. It is therefore advisable to use a relatively unstable inducer that causes small perturbations to

the training set. The weak learners are also homogeneous, i.e., they are of the same model type, such as all being trees for example. There is also an added benefit of parallelizing the process since classifiers are trained on independent samples. In the end, an average for regression, or majority prediction for classification, is used as the output. This majority voting for classification is known as hard voting or majority voting.

1.1. Bagging vs Boosting

We've touched on boosting in Module 3, and we will go through it in more detail in Lessons 3 and 4 of this module. At a high level, it is worth pausing and reflecting on the differences between these two methods. Figure 1 illustrates a summary of the methodology of Bagging and Boosting. Bagging is a parallel process whereby each base learner is not dependent on another, and the dataset for each is also independently sampled. Boosting on the other hand is done sequentially such that each base learner is developed based on the previous learner. With boosting, the training dataset, or samples fed into the base learner, is derived from the predictions of the previous base learner.

Figure 1: Bagging vs Boosting

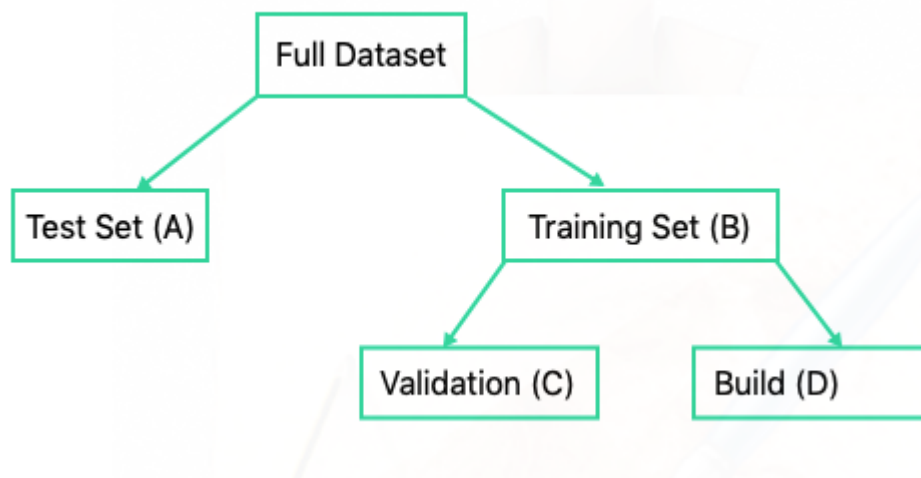


Base learner icon has been designed using resources from Flaticon.com

The downfall of an ensemble learner is the lack of interpretation of the model. It is not clear as to the decision flow through features as in a decision tree classifier since it is various classifiers that resulted in the final decision. There are methods such as **SHAP** and **LIME** that have overcome this problem and removed the black-box element of ensemble models, however accurate methods like SHAP are computationally expensive and can take a long time especially as observations that require interpreting increases. We look at another ensemble learning method called *stacking* in the next section.

2. Stacking

Figure 2: Datasets for Building a Model



To understand stacking, we first need to understand the validation data and model build set. Figure 2 shows a full dataset commonly split between a test and training set shown by A and B respectively. This is often done in Python with the `train_test_split` function from `sklearn`. We will assume we are working with a binary target with class 0 and 1. To provide some context, let's assume our full dataset has 10,000 observations. We split our data into 75/25 Train/Test split, so A and B have 2,500 and 7,500 observations respectively. The training set, B, is then further split into a 20/80 split, comprising the validation (C) and build (D) set, such that C has 1,500 observations and D has 6,000 observations. Dataset D is used to train a model, and the model's performance is assessed on C. Keep in mind that dataset D is also commonly referred to as the training set since we train our classifiers on this set; however, we use *Build* to distinguish this from B. We can use C and D to build the model since it allows for fine-tuning a model and reduces the chances of overfitting.

With these datasets in mind, let's suppose we train 3 classifiers such as Naive Bayes (Clf1), SVM (Clf2), and a decision tree (Clf3) on D. We then apply these classifiers on the validation set, C, and obtain the following:

```
In [2]: df = pd.read_csv("meta_dat2train.csv", index_col=False)
df
```

Out[2]:

	Clf1	Clf2	Clf3	Y
0	1	1	0	0
1	0	0	1	1
2	0	0	0	0
3	1	1	0	0
4	0	0	1	1
...	...	...	...	...
1495	0	0	1	1
1496	0	0	0	0
1497	1	1	0	0
1498	0	0	1	1
1499	0	0	0	0

1500 rows × 4 columns

The last column "Y" is the actual class value. Looking at the dataset above, we can use columns Clf1 to Clf3 as inputs into another model with target variable Y. This model that we develop is called a *Meta* or *Level-1* model and the approach described is called *blending*. Datasets C and D can be randomly shuffled into different 20/80 arrangements say k . A term to describe this is called *k-fold cross-validation*. This can be shown in Figure 3. We can develop the Meta model on the k^{th} bucket and use the remaining $k - 1$ buckets to train the 3 classifiers, Clf1-3. This approach over k -folds is known as *stacking* and is the main difference between blending. The classifiers Clf1-3 are known as base models, and this does not have to be restricted to 3, as in our example. You'll notice that 3 different families of classifiers were used, which is different from bagging. This implies that base or individual learners for stacking can be heterogeneous.

Figure 3: K-Fold Cross-Validation



3. Conclusion

Ensemble learning is based on the principle that many weak learners combined provide a better estimate than a single learner. This method often improves the bias-variance tradeoff problem. We've mentioned two popular ensemble learning methods, namely Bagging and Stacking. Bagging is restricted to homogeneous learners whereas stacking is not. In the next lesson, we will implement an example of each method in Python.

References

- Ali, Kamal M., and Michael J. Pazzani. "Error Reduction through Learning Multiple Descriptions." *Machine Learning*, vol. 24, 1996, pp. 173-202.
- Opitz, David W., and Richard Maclin. "Popular Ensemble Methods: An Empirical Study." *Journal of Artificial Intelligence Research*, vol. 11, 1999, pp. 169-172. All rights reserved WQU WorldQuant University QQQQ

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. James, Gareth, et al. *An Introduction to Statistical Learning with Applications in R*. 2nd ed., Springer, 2021.

- **Required pages:** pp. 340–343
- **Estimated reading time:** 15 minutes

Note: This book is not open access; however, the publisher and authors have made a downloadable copy available online. Due to copyright issues, we cannot provide you with a direct link, but if you google the book's title, you should be able to find and download a copy of it.

MACHINE LEARNING IN FINANCE

MODULE 4 | LESSON 2

ENSEMBLE LEARNING IN PRACTICE

Reading Time

15 minutes

Prior Knowledge

Bagging, Stacking, SVM, Decision Trees, Naïve Bayes, Logistic regression, ROC curve, AUC.

Keywords

Base Learners, Meta Model.

In the previous lesson, we introduced two ensemble learning methods, namely bagging and stacking. This lesson will put these two methods into practice using Python. All rights reserved WQU WorldQuant University QQQQ

1. Introduction

In this predictive problem, we apply ensemble methods to predict whether the Luxembourg index (LUXXX) exceeds a return of 0.25% in any direction. The predictors are a combination of country indices and technical indicators. Let's begin by importing the data.

```
In [1]: import warnings

import numpy as np
import pandas as pd

warnings.filterwarnings("ignore")

data_df = pd.read_csv("MScFE 650 MLF GWP Data.csv")
# Convert string to datetime
data_df["Date"] = pd.to_datetime(data_df["Date"])
```

We create the target variable as before.

```

In [2]: # Set Target Index for predicting
target ETF = "LUXXX"

# Use returns instead of prices for other Indices
# Other Indices used as Index_features
ETF_features = data_df.loc[:, ~data_df.columns.isin(["Date",
target ETF])].columns
data_df[ETF_features] = data_df[ETF_features].pct_change()

data_df[target ETF + "_returns"] = data_df[target ETF].pct_change()

# Create Target Column.
# Shift period for target column
data_df[target ETF + "_returns" + "_Shift"] = data_df[target ETF +
"_returns"].shift(-1)

# Strategy to take long position for anticipated returns of 0.5%
data_df["Target"] = np.where(
    (data_df[target ETF + "_returns_Shift"].abs() > 0.025), 1, 0
)

```

Create the predictors below, namely four country indices and three technical indicators: slow to fast moving average ratio (SMA_ratio), relative strength index (RSI), and rate of change (RC).

```

In [3]: # Four country indices used.
feats = ["MSCI KOREA", "MSCI DENMARK", "MSCI FRANCE", "MSCI NORWAY"]

# creating the technical indicators
data_df["SMA_5"] = data_df[target ETF].rolling(5).mean()
data_df["SMA_15"] = data_df[target ETF].rolling(15).mean()
data_df["SMA_ratio"] = data_df["SMA_15"] / data_df["SMA_5"]

# Can drop SMA columns since not needed anymore.
data_df.drop(["SMA_5", "SMA_15"], axis=1, inplace=True)

# shift the price of the target by 1 unit previous in time
data_df["Diff"] = data_df[target ETF] - data_df[target ETF].shift(1)
data_df["Up"] = data_df["Diff"]
data_df.loc[(data_df["Up"] < 0), "Up"] = 0

data_df["Down"] = data_df["Diff"]
data_df.loc[(data_df["Down"] > 0), "Down"] = 0
data_df["Down"] = abs(data_df["Down"])

data_df["avg_5up"] = data_df["Up"].rolling(5).mean()
data_df["avg_5down"] = data_df["Down"].rolling(5).mean()

data_df["avg_15up"] = data_df["Up"].rolling(15).mean()
data_df["avg_15down"] = data_df["Down"].rolling(15).mean()

data_df["RS_5"] = data_df["avg_5up"] / data_df["avg_5down"]
data_df["RS_15"] = data_df["avg_15up"] / data_df["avg_15down"]

data_df["RSI_5"] = 100 - (100 / (1 + data_df["RS_5"]))
data_df["RSI_15"] = 100 - (100 / (1 + data_df["RS_15"]))

data_df["RSI_ratio"] = data_df["RSI_5"] / data_df["RSI_15"]

# Can drop RS Calc columns
data_df.drop(
    ["Diff", "Up", "Down", "avg_5up", "avg_5down", "avg_15up",
     "avg_15down"],
    axis=1,
    inplace=True,
)

data_df["RC"] = data_df[target ETF].pct_change(15)

# all_feats
feats.append("SMA_ratio")
feats.append("RSI_ratio")
feats.append("RC")

```

Now that we have our data, we can apply the ensemble methods beginning with bagging.

2. Bagging with Random Forest

An example of a classifier that uses the bagging algorithm is the random forest classifier. Random forest is a tree-based algorithm; therefore, the weak learners

are decision trees. The "random" applies more to the random features allocated to each decision tree; hence, not all features are used by the weak learners. The different subset of features and different samples used by the decision trees also result in uncorrelated trees, which improves the performance of the algorithm. The final outcome is simply a majority vote among the weak learners. For instance, if there are five decision trees in the random forest and three trees predict class 1 while two trees predict class 0, the random forest classification would be class 1 due to the majority prediction. We begin by importing the necessary libraries.

```
In [4]: from sklearn import model_selection
from sklearn.ensemble import RandomForestClassifier
```

Perform the train/test split of 80/20.

```
In [5]: # Train/Test split
# Train/Test split. No NaNs in the data.
NoNaN_df = data_df.dropna()
X = NoNaN_df[feats]

X = X.iloc[:, :] # .values
y = NoNaN_df.loc[:, "Target"] # .values

del NoNaN_df

# from sklearn.cross_validation import train_test_split
X_train, X_test, y_train, y_test = model_selection.train_test_split(
    X, y, test_size=0.2, random_state=0
)
```

We now create the random forest classifier. We call it our bag model since it is our bagging classifier. Among the hyperparameters specified is `n_estimators`, which specifies the number of trees. To reduce the time taken to train the classifier, we use a small number of 10 trees. A large number of trees will not lead to overfitting according to James (341).

```
In [6]: bagmodel = RandomForestClassifier(n_estimators=10, random_state=4)
```

We train the model and look at its accuracy on the train and test set.

```
In [7]: bagmodel.fit(X_train, y_train)

print("Accuracy on train set: %0.4f" % (bagmodel.score(X_train, y_train)))
print("Accuracy on test set: %0.4f" % (bagmodel.score(X_test, y_test)))
```

Accuracy on train set: 0.9787

Accuracy on test set: 0.5208

The big difference between the accuracies on the train and test set indicate over-fitting. We will look at optimal hyperparameter values to improve performance such as `min_samples_split`, which is the minimum number of samples required to split an internal node. We've mentioned `max_depth` in Lesson 3 of Module 3 for decision trees, which is the same for random forests.

With limited computational power and quick turnaround times, we will explore just a few samples for these parameters. We use `GridSearchCV` as in Lesson 2 of Module 5 to find the optimal hyperparameters.

```
In [8]: from sklearn.model_selection import GridSearchCV
```

```
In [9]: # defining parameter range
param_grid = {
    "n_estimators": [10],
    "max_depth": [2, 3, 4],
    "min_samples_split": [2, 4, 8],
}

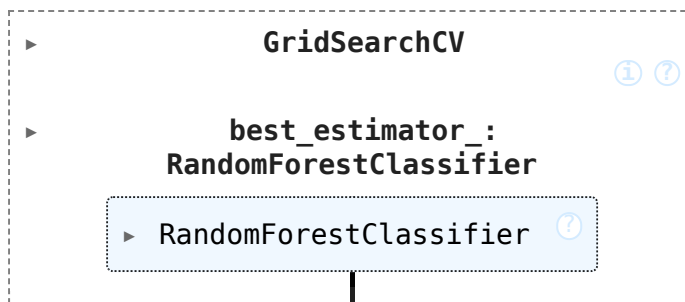
grid = GridSearchCV(
    RandomForestClassifier(random_state=8), param_grid, refit=True,
    verbose=3, cv=3
)

# fitting the model for grid search
grid.fit(X_train, y_train)
```

Fitting 3 folds for each of 9 candidates, totalling 27 fits

```
[CV 1/3] END max_depth=2, min_samples_split=2, n_estimators=10;; score=0.698
total time= 0.0s
[CV 2/3] END max_depth=2, min_samples_split=2, n_estimators=10;; score=0.651
total time= 0.0s
[CV 3/3] END max_depth=2, min_samples_split=2, n_estimators=10;; score=0.661
total time= 0.0s
[CV 1/3] END max_depth=2, min_samples_split=4, n_estimators=10;; score=0.698
total time= 0.0s
[CV 2/3] END max_depth=2, min_samples_split=4, n_estimators=10;; score=0.651
total time= 0.0s
[CV 3/3] END max_depth=2, min_samples_split=4, n_estimators=10;; score=0.661
total time= 0.1s
[CV 1/3] END max_depth=2, min_samples_split=8, n_estimators=10;; score=0.698
total time= 0.0s
[CV 2/3] END max_depth=2, min_samples_split=8, n_estimators=10;; score=0.635
total time= 0.0s
[CV 3/3] END max_depth=2, min_samples_split=8, n_estimators=10;; score=0.694
total time= 0.0s
[CV 1/3] END max_depth=3, min_samples_split=2, n_estimators=10;; score=0.683
total time= 0.1s
[CV 2/3] END max_depth=3, min_samples_split=2, n_estimators=10;; score=0.619
total time= 0.0s
[CV 3/3] END max_depth=3, min_samples_split=2, n_estimators=10;; score=0.677
total time= 0.0s
[CV 1/3] END max_depth=3, min_samples_split=4, n_estimators=10;; score=0.683
total time= 0.0s
[CV 2/3] END max_depth=3, min_samples_split=4, n_estimators=10;; score=0.619
total time= 0.1s
[CV 3/3] END max_depth=3, min_samples_split=4, n_estimators=10;; score=0.661
total time= 0.0s
[CV 1/3] END max_depth=3, min_samples_split=8, n_estimators=10;; score=0.683
total time= 0.0s
[CV 2/3] END max_depth=3, min_samples_split=8, n_estimators=10;; score=0.619
total time= 0.0s
[CV 3/3] END max_depth=3, min_samples_split=8, n_estimators=10;; score=0.629
total time= 0.1s
[CV 1/3] END max_depth=4, min_samples_split=2, n_estimators=10;; score=0.730
total time= 0.0s
[CV 2/3] END max_depth=4, min_samples_split=2, n_estimators=10;; score=0.619
total time= 0.0s
[CV 3/3] END max_depth=4, min_samples_split=2, n_estimators=10;; score=0.726
total time= 0.0s
[CV 1/3] END max_depth=4, min_samples_split=4, n_estimators=10;; score=0.746
total time= 0.1s
[CV 2/3] END max_depth=4, min_samples_split=4, n_estimators=10;; score=0.619
total time= 0.0s
[CV 3/3] END max_depth=4, min_samples_split=4, n_estimators=10;; score=0.677
total time= 0.0s
[CV 1/3] END max_depth=4, min_samples_split=8, n_estimators=10;; score=0.714
total time= 0.0s
[CV 2/3] END max_depth=4, min_samples_split=8, n_estimators=10;; score=0.635
total time= 0.1s
[CV 3/3] END max_depth=4, min_samples_split=8, n_estimators=10;; score=0.710
total time= 0.0s
```

Out[9]:



```
In [10]: # print best parameter after tuning
print(grid.best_params_)
```

```
# print how our model looks after hyper-parameter tuning
print(grid.best_estimator_)
```

```
{'max_depth': 4, 'min_samples_split': 2, 'n_estimators': 10}
RandomForestClassifier(max_depth=4, n_estimators=10, random_state=8)
```

Apply optimal parameters and check if test score improves.

```
In [11]: # Train with Tuned RF
# Create a tuned RF Classifier
bagmodel_tuned = RandomForestClassifier(
    n_estimators=10,
    max_depth=grid.best_params_["max_depth"],
    min_samples_split=grid.best_params_["min_samples_split"],
    random_state=8,
)

bagmodel_tuned.fit(X_train, y_train)
print("Accuracy on test set: %0.4f" % (bagmodel_tuned.score(X_test,
y_test)))
```

Accuracy on test set: 0.5833

There is an uplift in accuracy for the test set by just exploring a small hyperparameter space. The small hyperparameter space is considered to keep computational time within reasonable limits. Students are encouraged to explore more hyperparameter options. Another measure of model performance to explore is the out-of-bag error, which is essentially the test error of a bagged model (James 342). Keep in mind that accuracy is based on the predicted classes, whereas the ROC AUC is based on predicted scores. We therefore plot the ROC Curve as in the previous modules to compare to the random guess or no-skill model. From the output below, there is a clear benefit to the Random forest model over the no-skill model.

```
In [12]: import matplotlib.pyplot as plt

# Performance
from sklearn.metrics import roc_auc_score, roc_curve

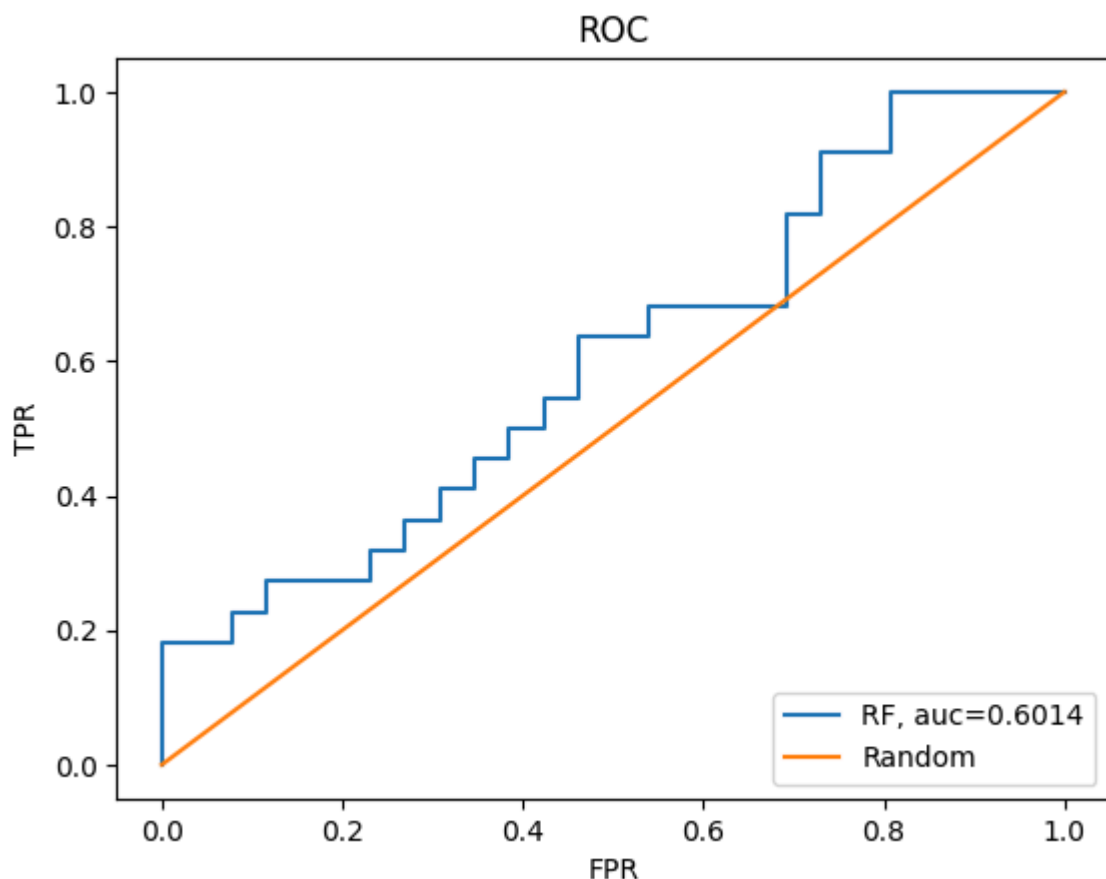
# predicted probabilities generated by tuned classifier
y_pred_proba = bagmodel_tuned.predict_proba(X_test)

# RF ROC dependencies
fpr, tpr, _ = roc_curve(y_test, y_pred_proba[:, 1])
auc = round(roc_auc_score(y_test, y_pred_proba[:, 1]), 4)

# RF Model
plt.plot(fpr, tpr, label="RF, auc=" + str(auc))

# Random guess model
plt.plot(fpr, fpr, "-", label="Random")
plt.title("ROC")
plt.ylabel("TPR")
plt.xlabel("FPR")

plt.legend(loc=4)
plt.show()
```



Another useful feature from bagging is the *Out of Bag Score*. Refer to the video below for an explanation on this feature.

```
In [13]: from IPython.display import VimeoVideo

VimeoVideo("785136013", h="2e436e22bd", width=600)
```

Out[13]:

[Access Video Transcript here](#)

3. Stacking Model

The stacking model depends on the base models and meta model. For this illustration, we use the Gaussian Naïve Bayes, decision tree, and SVM classifiers as the base models. For the meta model, we use the simplistic logistic regression model. This is to show how we can leverage the logistic regression model using stacking. Let's import the necessary libraries for each of them. The names are well described and should be easy to match to the respective model.

```
In [14]: from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
```

We define three base classifiers without any hyperparameter specifications, i.e., we use the default settings other than the kernel for SVC. This is to show the predictive power of stacking without optimizing.

```
In [15]: clf1 = DecisionTreeClassifier(random_state=2) # Decision Tree
clf2 = SVC(kernel="rbf", random_state=2) # Support Vector Classifier
clf3 = GaussianNB() # Gaussian Naive Bayes
est_rs = [("DTree", clf1), ("SVM", clf2), ("NB", clf3)]
```

Define the meta model.

```
In [16]: mylr = LogisticRegression(random_state=2)
```

```
In [17]: # creating a stacking classifier
stackingCLF = StackingClassifier(
    estimators=est_rs, final_estimator=mylr, stack_method="auto", cv=3
)
```

The stacking model is ready to go, and now, we need to train it on the training data. We also show the training score since we want to compare it to each of its base learners below as well.

```
In [18]: stackingCLF.fit(X_train, y_train)
acc_score = stackingCLF.score(X_train, y_train)
round(acc_score, 4)
```

```
Out[18]: 0.7021
```

```
In [19]: # Let's run the individual base learners and stacking `clf` to compare
for iterclf, iterlabel in zip([clf1, clf2, clf3], ["DTree", "SVM", "NB"]):
    scores = model_selection.cross_val_score(
        iterclf, X_train, y_train, cv=3, scoring="accuracy"
    )
    print("accuracy: %0.3f %s " % (scores.mean(), iterlabel))
```

```
accuracy: 0.559 DTree
```

```
accuracy: 0.697 SVM
```

```
accuracy: 0.686 NB
```

It is quite evident that the stacking model outperforms all of its base learners and shows the benefit of ensemble learning in this case. It is worth comparing the performance of the two ensemble learning models used in this lesson; therefore, we look at the ROC curve and AUC of the stacking model versus the random forest model.

```
In [20]: # predicted probabilities generated by tuned classifier
y_pred_probaStack = stackingCLF.predict_proba(X_test)

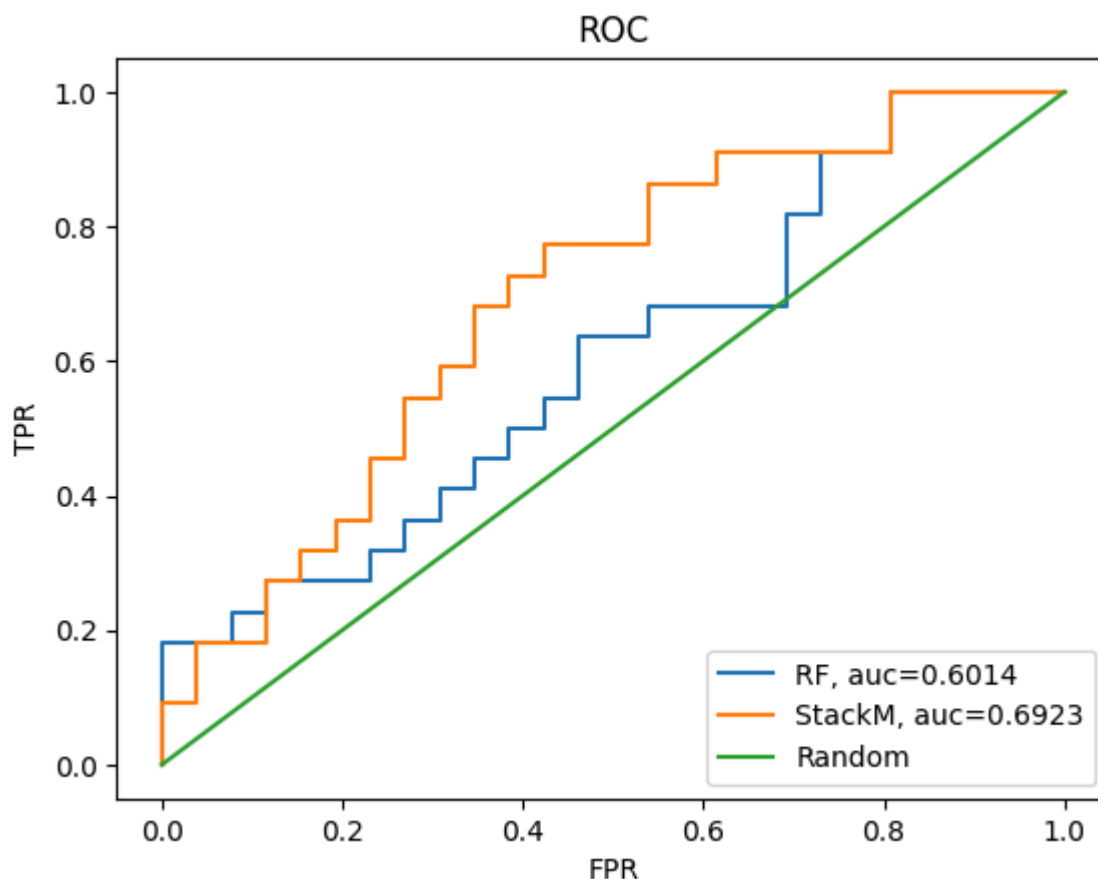
# Stacking Model ROC dependencies
fpr, tpr, _ = roc_curve(y_test, y_pred_probaStack[:, 1])
auc = round(roc_auc_score(y_test, y_pred_probaStack[:, 1]), 4)

# RF ROC dependencies
fpr_RF, tpr_RF, _ = roc_curve(y_test, y_pred_proba[:, 1])
auc_RF = round(roc_auc_score(y_test, y_pred_proba[:, 1]), 4)

# RF Model
plt.plot(fpr_RF, tpr_RF, label="RF, auc=" + str(auc_RF))
# Stacking Model
plt.plot(fpr, tpr, label="StackM, auc=" + str(auc))

# Random guess model
plt.plot(fpr, fpr, "-", label="Random")
plt.title("ROC")
plt.ylabel("TPR")
plt.xlabel("FPR")

plt.legend(loc=4)
plt.show()
```



From the plot above, it is clear that the stacking model slightly outperforms the random forest model.

4. Conclusion

This lesson looked at bagging and stacking in practice to predict a change of the LUXXX index by a certain threshold, i.e., a classification problem. Both these methods show added value over a no-skill model. In the next lesson, we will cover another ensemble learning method namely boosting and adaptive boosting.

References

James, Gareth et al. *An Introduction to Statistical Learning: With Applications in R*. 2nd ed., Springer, 2021.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Chuan, Yijian, et al. "The Success of AdaBoost and Its Application in Portfolio Management." *arXiv*, 2021, <https://arxiv.org/abs/2103.12345>.

- **Required pages:** pp. 1–5
- **Estimated reading time:** 15 minutes

MACHINE LEARNING IN FINANCE

MODULE 4 | LESSON 3

BOOSTING METHODS

Reading Time	25 minutes
Prior Knowledge	Classification, Decision Trees
Keywords	Sampling with replacement, Performance, Amount of Say

The previous lesson covered ensemble learning methods such as bagging and stacking and also applied these methods to a problem in finance. This lesson will cover another ensemble learning method called adaptive boosting in detail.

1. Introduction

Boosting was briefly discussed in Module 3, Lesson 3, and we will add to that in this lesson. Boosting, like a random forest, uses many weak learners to improve the end result. Remember that a weak learner is a learner that performs slightly better than a no-skill model. The difference with boosting and a random forest model is that the weak learners are **not** independent. These learners are built sequentially, i.e., a weak learner "improves" upon the learner before it. This improvement happens with the next learner focusing on the previous weak learner's errors. This is the fundamental concept of boosting. This lesson will introduce the reader to *adaptive* boosting.

2. Adaptive Boosting

Adaptive boosting or AdaBoost is a popular algorithm and was among the earliest boosting models introduced. The sequential development of this model occurs by adjusting the weights or importance of the previous learner's predictions. Higher weights are given to predictions that deviate from the actual value and lower weights to the predictions that are close to the actual value. In other words, more weighting is assigned to incorrect predictions, or errors are corrected iteratively. AdaBoost can work on both classification and regression problems. One should keep in mind that AdaBoost is not restricted to tree-based algorithms; however, the weak learners or base learners are homogeneous in that they must all be of the same family or class of models. As with random forests, the majority vote is the final decision. However, with AdaBoost, not all

base learners have equal weight or "say" on the vote. We will use the Taiwan Credit Card default dataset to explain how AdaBoost works.

```
In [1]: # import any libraries needed to create dataset
import random
import warnings

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn import tree

# from sklearn.metrics import confusion_matrix
from sklearn.tree import DecisionTreeClassifier # ,
`DecisionTreeRegressor`

warnings.filterwarnings("ignore")
```

The dataset is used to identify default payments for credit cards in Taiwan. The Target class is binary such that 1 refers to a client that defaults on their credit repayments and 0 otherwise. There are 22 predictors in this dataset. More information on this dataset can be found at <https://archive.ics.uci.edu/ml/datasets/default+of+credit+card+clients> . We import the dataset below.

```
In [2]: data4lesson = pd.read_excel("Copy of default of credit card clients.xlsx")
```

```
In [3]: # Make ID column the index
data4lesson.set_index("ID", inplace=True, drop=True)
data4lesson.head()
```

Out[3]:

	LIMIT BAL	SEX	EDUCATION	AGE	PAY 0	PAY 2	PAY 3	PAY 4	PAY 5	PAY 6	..	BILL 4	BILL 5	BILL 6	PAY 1	PAY 2	PAY 3	PAY 4	PAY 5	PAY 6	default payment next month
ID																					
1	20000	1	2	24	2	-1	-1	-2	-2	...	0	0	0	0	689	0	0	0	0	0	1
2	120000	1	2	26	-1	2	0	0	0	2	...	3272	3455	3261	0	1000	1000	1000	0	2000	1
3	90000	1	2	34	0	0	0	0	0	0	...	1431	1498	1559	1558	1500	1000	1000	1000	5000	0
4	50000	1	2	37	0	0	0	0	0	0	...	28314	28959	29547	2000	2019	1000	1000	1000	1000	0
5	50000	0	2	57	-1	0	-1	0	0	0	...	20940	19144	19931	2000	36681	1000	9000	6899	6799	0

5 rows × 23 columns

```
In [4]: # Rename the target column to 'Class'
data4lesson.rename(columns={"default payment next month": "Class"},
inplace=True)

# keep predictors in a separate list
feats = list(data4lesson.columns[0 : data4lesson.shape[1] - 1])
```

Check the dimension of the data and the frequency of each target class.

```
In [5]: print("Rows {} columns {}".format(data4lesson.shape[0],
      data4lesson.shape[1]))
      data4lesson["Class"].value_counts()
```

Rows 30000 columns 23

```
Out[5]: Class
0      23364
1       6636
Name: count, dtype: int64
```

Let's begin the steps for AdaBoost.

2.1 Step 1: Sample Weights

The first step for AdaBoost is to perform random sampling with replacement on our dataset. To do this, we initially assign each observation an equal weight, i.e., $1/N$ for N being the number of observations. To explain the process of random sampling, we will insert a cumulative column of the weights called

`RangeSelect` to illustrate the idea.

```
In [6]: # Assign equal weights to all samples
N = len(data4lesson)
data4lesson["probR1"] = 1 / (1.0 * N)
data4lesson["RangeSelect"] = data4lesson["probR1"].cumsum()
# Show last 5 columns
data4lesson[["PAY_AMT5", "PAY_AMT6", "Class", "probR1",
"RangeSelect"]].head()
```

```
Out[6]:
```

	PAY_AMT5	PAY_AMT6	Class	probR1	RangeSelect
ID					
1	0	0	1	0.000033	0.000033
2	0	2000	1	0.000033	0.000067
3	1000	5000	0	0.000033	0.000100
4	1069	1000	0	0.000033	0.000133
5	689	679	0	0.000033	0.000167

Now that we have the cumulative column, we are able to explain the method of sampling.

2.2 Step 2: Sampling with Replacement

Think of each value in the `RangeSelect` column as an upper limit with the lower limit being the value in the observation, or row, before it. For example, the output above ID 2 would have a `RangeSelect` of $[0.000033, 0.000067)$. Now draw a random number in the range $[0, 1]$. Suppose this number is 0.000043. We therefore add ID 2 to our sample since the random number of 0.000043 is in

the range $[0.000033, 0.000067]$, which corresponds to ID 2. If our random number drawn was 0.00014, then ID 5 would be added to the sample since 0.00014 is in the range $[0.000133, 0.000167]$. One may notice that observations with large ranges would have a higher chance of being selected. Hence, large weights or `probR1` values increase the chances of being sampled, which is crucial for the AdaBoost algorithm. Fortunately, Python has a more elegant and efficient way of performing random sampling with replacement, so we won't need the `RangeSelect` column anymore.

```
In [7]: # Drop `RangeSelect` column
data4lesson.drop("RangeSelect", axis=1, inplace=True)
```

We'll use the pandas `sample` function to sample with replacement based on the weights column `probR1`. The amount of samples is the same as N, that is, the same number of observations as our original dataset. There will be duplicate observations in this new dataset since the sampling is done with replacement.

```
In [8]: # Random selection based on weights
# specify seed in case we wish to replicate
# the Random Sampling with replacement
random.seed(10)

data4lesson1 = data4lesson.sample(
    len(data4lesson), replace=True, weights=data4lesson["probR1"]
)
```

2.3 Step 3: Fit a Base Learner

The next step in AdaBoost is to fit a weak classifier or base learner to the samples. For this example, we will use decision trees as our base learners. We'll call this learner `weak_l1` and fit it to the samples.

```
In [9]: # Since it is a stump the max depth is 1
weak_l1 = DecisionTreeClassifier(random_state=10, max_depth=1)

X = data4lesson1.loc[:, feats]
y = data4lesson1["Class"]

weak_l1.fit(X, y)
```

```
Out[9]: ▼ DecisionTreeClassifier
DecisionTreeClassifier(max_depth=1, random_state=10)
```

We should be able to view the flow diagram of the stump.

```

In [10]: # visualize the stump
# features we used

fn = feats

# labels of the target class

cn = ["0", "1"]

fig = plt.figure(figsize=(25, 20))
_ = tree.plot_tree(weak_l1, feature_names=fn, class_names=cn, filled=True)

# save the figure to file

# fig.savefig('imagename.png')
fig.savefig("decistion_tree.png")

```

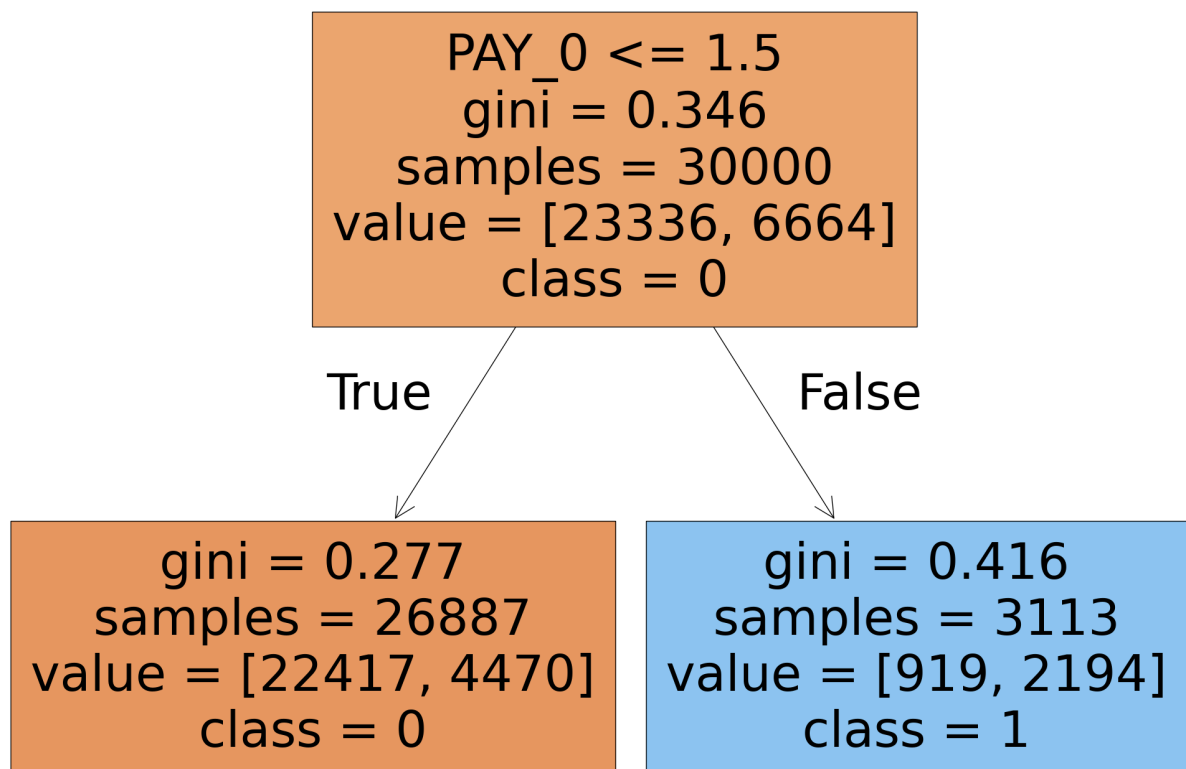


Figure 1: The Decision Stump of the Base Learner at the First Iteration

Notice the base learner found *Pay_0* to be the strongest predictor to split on. Let us compare the predictions to the actual values.

```

In [11]: data4lesson1["pred1"] = weak_l1.predict(X)

```

```

In [12]: data4lesson1[["Class", "pred1", "probR1"]].head()

```

Out[12]:

	Class	pred1	probR1
ID			
24086	0	0	0.000033
14550	1	0	0.000033
12041	0	0	0.000033
8694	0	0	0.000033
13362	0	0	0.000033

2.4 Step 4: Calculate Performance

The snapshot of just the first five rows shows one misclassification for ID 16973. The number of misclassifications will determine the *amount of say* or *performance* of this classifier. The formula for the amount of say is

$$\text{Amount of say} = \frac{1}{2} \ln \left(\frac{1 - \text{Total Error}}{\text{Total Error}} \right) \quad (1.1)$$

where the **Total Error** is simply the sum of weights of misclassified observations. The Total Error for the base classifier at our first iteration, i.e., `weak_l1` is

```
In [13]: # Total error of weak_l1
E_1 = round(
    np.sum(
        np.where(
            data4lesson1["Class"] != data4lesson1["pred1"],
            data4lesson1["probR1"], 0
        )
    ),
    5,
)

E_1
```

Out[13]: np.float64(0.17963)

The Amount of say in (1.1) against the total error in general is shown below.

```
In [14]: n_pts = 100
TE = 0.01 + np.arange(n_pts) * (1 - 0.01) / n_pts
AoS = 0.5 * np.log((1 - TE) / TE)
plt.plot(TE, AoS)
plt.ylabel("Amount of Say")
plt.xlabel("Total Error")
```

Out[14]: Text(0.5, 0, 'Total Error')

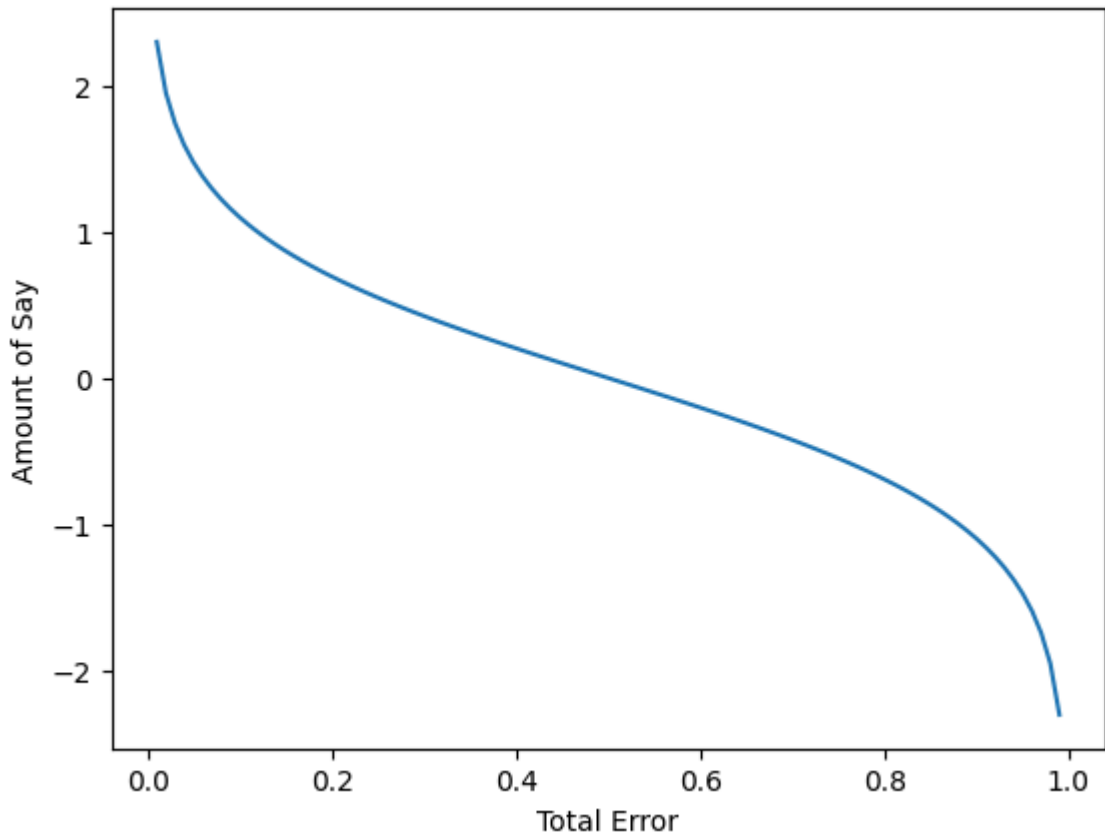


Figure 2: Performance or Amount of Say against Total Error for a Classifier

From figure 2, we see that for close to perfect classifications, the base classifier will have a performance that tends to infinity. For almost all incorrect classifications, the learner will have performance that tends to negative infinity. For a Total Error of 0.5, this results in a performance of zero. If all samples had equal weights, this would be equivalent to a no-skill classifier. In practice, a small error term is added to the Total Error to avoid the case when the Total Error is zero. Our first base learner has a Total Error of 0.1796, which results in a performance of 0.7596. Remember that AdaBoost focuses on the misclassifications by adjusting the weights in an iterative manner. Let us now look into this process of adjusting the weights.

For the observations that were misclassified, the formula to adjust the weights is given by,

$$\text{Adjusted weight} = \text{Current Weight} \times e^{\text{performance}} \quad (1.2)$$

similarly the formula for adjusting the weights of the correct classification is,

$$\text{Adjusted weight} = \text{Current Weight} \times e^{-\text{performance}} \quad (1.3)$$

Notice the difference between (1.2) and (1.3) is the sign of the exponent. This implies that correct classifications are downweighted whereas misclassifications are adjusted to larger weights. Applying (1.2) and (1.3) to the weights creates the new weights shown below.

```
In [15]: # performance of weak_l1
p1 = 0.5 * np.log((1 - E_1) / E_1)
# Call the new weights `probR2` since it is the second iteration
data4lesson1["probR2"] = np.where(
    data4lesson1["Class"] != data4lesson1["pred1"],
    data4lesson1["probR1"] * np.exp(p1), # misclassifications (1.2)
    data4lesson1["probR1"] * np.exp(-p1),
)
```

```
In [16]: data4lesson1[["Class", "pred1", "probR1", "probR2"]].head()
```

```
Out[16]:
```

	Class	pred1	probR1	probR2
ID				
24086	0	0	0.000033	0.000016
14550	1	0	0.000033	0.000071
12041	0	0	0.000033	0.000016
8694	0	0	0.000033	0.000016
13362	0	0	0.000033	0.000016

We can see the correct classifications have new weights that are lower than the original weights of 3.3×10^{-5} whereas the misclassifications have weights that have increased. Remember that an increase in weight would increase the chances of choosing that observation when sampling with replacement. One thing to take note of is that the new weights are normalized such that they sum to 1 as were the original weights in column `probR1`.

```
In [17]: # normalize the weights
data4lesson1["probR2"] = data4lesson1["probR2"] /
data4lesson1["probR2"].sum()
```

This brings us back to Step 1 with a new set of weights to repeat steps 2 to 4. The number of times we would repeat these cycles would depend on the number of weak classifiers we specify. This is the iterative process that AdaBoost follows to develop the final model. Ideally, we would want to continue this process until all observations are classified correctly, but this can allow for overfitting. Hyperparameter tuning should be taken into consideration. To illustrate the final prediction of the model, let's assume we had chosen 5 base learners and they each had the following performance or Amount of Say and prediction.

```
In [18]: dat = [[0.7596, 1], [0.7821, 1], [0.79019, 1], [0.8241, 0], [0.8410, 0]]
model_summary = pd.DataFrame(data=dat, columns=["performance",
"prediction"])
model_summary.index.name = "iteration"
model_summary
```

Out[18]:

	performance	prediction
iteration		
0	0.75960	1
1	0.78210	1
2	0.79019	1
3	0.82410	0
4	0.84100	0

The hypothetical result above over 5 iterations shows 3 base learners predicting class 1 and 2 base learners predicting class 0. The final vote is based on the sum of performance for each class voted, i.e., instead of the counts being used, the sum of the performance is used. The calculation below shows that class 1 would be favored since the sum of the performance for base learners in favor of that class is 2.332, which is greater than the sum of the performance for base learners in favor of class 0.

```
In [19]: class_0 = np.sum(
            np.where(model_summary["prediction"] == 0,
            model_summary["performance"], 0)
        )
        class_1 = np.sum(
            np.where(model_summary["prediction"] == 1,
            model_summary["performance"], 0)
        )

        print("Weight of Votes ")
        print("Class 0: {} Class 1: {}".format(class_0, class_1))
```

```
Weight of Votes
Class 0: 1.6651 Class 1: 2.33189
```

Something to keep in mind is that the example above showed the updating procedure on the entire dataset whereas in practice we would be using the training set to update and develop AdaBoost. We've also shown the process using weak decision stumps, but there is evidence to support more advanced learners being effective (Chuan et al. 3).

Boosting has found success in the data science field especially in Kaggle competitions. There is one more boosting technique we will cover in the next lesson called gradient boosting.

3. Conclusion

This lesson focused on adaptive boosting and showed the iterative updating procedure to derive the final model and prediction. The procedure was shown for a classification problem, namely predictive credit card payment defaults. In the next lesson, we'll look at another boosting algorithm called gradient boosting

and compare all ensemble learning methods covered in this module to a common problem.

References

1. Chuan, Yijian, et al. "The Success of AdaBoost and Its Application in Portfolio Management." *arXiv*, 2021, pp. 1-6.
2. University of California, Irvine Machine Learning Repository. "Default of Credit Card Clients."
<https://archive.ics.uci.edu/ml/datasets/default+of+credit+card+clients>

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Murphy, Kevin Patrick. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022,
<https://probml.github.io/pml-book/book1.html>.
- **Required pages:** Section 18.5 pages 609-610 and Section 18.5.5 pages 614-618
 - **Estimated reading time:** 20 minutes

MACHINE LEARNING IN FINANCE

MODULE 4 | LESSON 4

ENSEMBLE LEARNING COMPARISONS

Reading Time	25 minutes
Prior Knowledge	Boosting methodology, Adaptive Boosting, Ensemble learning, Derivatives
Keywords	Pseudo residuals, loss function, learning rate, gradient boosting

The previous lesson introduced the reader to the details of boosting, specifically adaptive boosting. This lesson will explore Gradient Boosting and compare all ensemble learning models covered in this module to a common predictive problem.

1. Gradient Boosting

In the previous lesson we've looked at AdaBoost as it was among the first designed boosting algorithm with a particular loss function that can be sensitive to outliers. Gradient Boosting, however, is a generic algorithm that allows the optimization of an arbitrary loss function, thus making Gradient Boosting more flexible and robust to outliers than AdaBoost. The requirement of the loss function is that it is differentiable. We've seen that AdaBoost places more emphasis on predictions that were incorrect from the previous base learner although there is still some weight assigned to the correct predictions. With Gradient Boosting, however, there is only an emphasis on building the subsequent base learners based on the misclassifications from the previous learner. To illustrate this, we will go through an example to explain the process. Unlike the previous section, we will look at a regression example, i.e., a continuous target variable to illustrate boosting applied to a different type of problem. The dataset we use is the house price dataset from the UCI Machine Learning Repository. We will begin by importing the necessary libraries for this lesson along with the data.

```
In [1]: # import all necessary libraries

import warnings

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.ensemble import (
    AdaBoostClassifier,
    GradientBoostingClassifier,
    RandomForestClassifier,
    StackingClassifier,
)
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB

# for stacking model later
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor

warnings.filterwarnings("ignore")
```

```
In [2]: data4gradboost = pd.read_excel("Real estate valuation data set.xlsx")

data4gradboost.set_index("No", drop=True, inplace=True)
data4gradboost.drop("X1 transaction date", axis=1, inplace=True)
data4gradboost.head()
```

```
Out[2]:
```

	X2 house age	X3 distance to the nearest MRT station	X4 number of convenie nce stores	X5 latitude	X6 longitud e	Y house price of unit area
No						
1	32.0	84.87882	10	24.98298	121.54024	37.9
2	19.5	306.59470	9	24.98034	121.53951	42.2
3	13.3	561.98450	5	24.98746	121.54391	47.3
4	13.3	561.98450	5	24.98746	121.54391	54.8
5	5.0	390.56840	5	24.97937	121.54245	43.1

Above shows a snapshot of the dataset, which has 5 predictors and a target variable in the last column that represents the price per unit area. Refer to <https://archive.ics.uci.edu/ml/datasets/Real+estate+valuation+data+set> for more details on this dataset. We start by separating the dataset into the features and target column for a train/test split. The size of the test set is not important

for this example since we are only interested in understanding the gradient boosting algorithm methodology.

```
In [3]: # Separate into X and Y
X = data4gradboost.iloc[:, :-1]
y = data4gradboost.iloc[:, -1]
```

```
In [4]: X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=2)
```

1.1 Step 1: Fit Weak Learner

Now that we have the data to develop our gradient boosting model, we can begin the first step. Similarly, for the AdaBoost in the previous lesson, we use weak decision tree classifiers as the base learners. We choose a `max_depth` of 2 and then fit the base learner to the training dataset.

```
In [5]: tree_1 = DecisionTreeRegressor(max_depth=2, random_state=2)
tree_1.fit(X_train, y_train)
```

```
Out[5]: ▾ DecisionTreeRegressor
DecisionTreeRegressor(max_depth=2, random_state=2)
```

1.2 Step 2: Calculate Residuals

Once we fit our first weak learner to the data, we then calculate the residuals. The residuals are simply given by

$$\text{residuals} = y_i - \hat{y}_i \quad (1)$$

where y is the actual outcome and $\hat{y}$ is the predicted value for observation i . The residuals now become the new target data to train the next base learner.

Gradient boost builds trees based on the residuals or errors of the previous tree.

1.3 Step 3: Train Next Base Learner on Residuals

```
In [6]: # predictions
y_pred1 = tree_1.predict(X_train)
# residuals become the next target data to train
y2_train = y_train - y_pred1
```

Once we have the new target values to train the next base learner, we continue this cycle, i.e., steps 1 to 3. In this example, we will do this for 3 base learners, i.e., 3 iterations.

```
In [7]: # initialize new tree or 2nd base learner
tree_2 = DecisionTreeRegressor(max_depth=2, random_state=2)
tree_2.fit(X_train, y2_train)

# predictions
y_pred2 = tree_2.predict(X_train)
# new target values
y3_train = y2_train - y_pred2

# initialize new tree or 3rd base learner
tree_3 = DecisionTreeRegressor(max_depth=2, random_state=2)
tree_3.fit(X_train, y3_train)

# last set of predictions. Stop at 3rd base learner
y_pred3 = tree_3.predict(X_train)
```

1.4 Final Step: Calculate Final Predictions

Finally, for predictions of unseen observations, we would add the predictions of each base learner as below. We assign the final predictions to `y_pred` and then calculate a performance metric, namely Root Mean Squared Error (RMSE). We obtain an RMSE of 9.9625.

```
In [8]: # Make predictions on test set for all 3 base learners
y1_pred = tree_1.predict(X_test)
y2_pred = tree_2.predict(X_test)
y3_pred = tree_3.predict(X_test)

y_pred = y1_pred + y2_pred + y3_pred

# check MSE
round(mean_squared_error(y_test, y_pred) ** 0.5, 4)
```

Out[8]: 9.9625

Building the gradient boosting algorithm from scratch is quite tedious, and it is easy to make an error coding it. Fortunately, Python has the built-in `GradientBoostingRegressor` from sklearn to make things much easier. Below is the more elegant and efficient way to code this. Notice the hyperparameter `learning_rate` = 1, which we'll address later.

```
In [9]: from sklearn.ensemble import GradientBoostingRegressor
```

```
In [10]: gbr = GradientBoostingRegressor(
          max_depth=2, n_estimators=3, random_state=10, learning_rate=1
          )
```

```
In [11]: gbr.fit(X_train, y_train)
```

Out[11]:

```
▼ GradientBoostingRegressor
GradientBoostingRegressor(learning_rate=1, max_depth=2,
n_estimators=3,
                        random_state=10)
```

The RMSE results are identical to our model built from scratch.

In [12]:

```
gbr_pred = gbr.predict(X_test)
round(mean_squared_error(y_test, gbr_pred) ** 0.5, 4)
```

Out[12]: 9.9625

The example above illustrates the procedure followed by a gradient boosting algorithm. Below will show how it works with a bit more attention to the mathematics behind it. All rights reserved WQU WorldQuant University QQQQ

1.5 Gradient Boosting Reasoning

The algorithm begins with a naive guess of using the average, say $\langle y \rangle$, for all house prices or target values. The reasoning behind this is gradient boosting aims to minimize the loss function $L(y_i, \hat{y}_i)$. A common L used is of the form

$$L(y_i, \hat{y}_i) = \frac{1}{2} \sum_i^N (y_i - \hat{y}_i)^2 \quad (1.1)$$

over N observations. This type of loss function is an L2 Loss function.

At the start, we assume $\hat{y}_i$ is a constant function $F_0(x)$ over predictors x . From calculus, minimizing implies finding the derivative of (1.1) with respect to $F_0(x)$, equating to zero and solving for $F_0(x)$. If we do this, the solution $F_0^*(x)$ is,

$$F_0^*(x) = \frac{1}{N} \sum_i^N y_i = \bar{y} \quad (1.2)$$

which is how we get the average as the first guess. Note, had we chosen a different L this may not have been the case. For illustrative purposes, we will use the loss function in (1.1) to explain the concept.

Remember from our example in section 1.1 that the next base learner would aim to predict the residuals of the previous base learner instead of the actual values. Starting off with predictions as the average of the actuals is a very simplistic estimation; however, the next iteration 1 will improve on the previous base learner predictions such that new predictions are

$$F_1(x) = F_0(x) + h_1(x) \quad (1.3)$$

where $h_1(x)$ is an added estimator such that together with the previous estimate F_0 , will improve the estimates. We hope that our new estimates would

predict the actuals i.e.,

$$F_1(x) = F_0(x) + h_1(x) = y \quad (1.4)$$

but rearranging (1.4) gives us $h_1(x) = y - F_0(x)$. This is just the residuals from the previous base learner's predictions and is what we were doing in section 1.1. So to get this estimator $h_1(x)$ we would provide as a target column $(y - F_0(x))$ and fit a base learner on the dataset $\{(x_i, y_i - F_0(x))\}_{i=1}^N$ or $\{(x_i, r_{i1})\}_{i=1}^N$ where r_{i1} are the pseudo residuals at iteration 1. In general at iteration j , we have from (1.4),

$$F_j(x) = F_{j-1}(x) + h_j(x) \quad (1.5)$$

and we develop our estimator $h_j(x)$ in the same way. There is a parameter ν , that is not shown in (1.5) where,

$$F_j(x) = F_{j-1}(x) + \nu h_j(x) \quad (1.6)$$

which we've set as $\nu = 1$ but can actually take on values in the range $[0, 1]$. This parameter is the *learning rate*, which improves accuracy in the long run. Think of it as a step size, and often many learners with small step sizes provide better results. From recursive substitution over all iterations, the last iteration T will provide a final model such that for a test or unseen data point x^* the prediction would be

$$F_T(x^*) = F_0 + \sum_j^{T-1} \nu h_j(x^*), \quad (1.6)$$

which ties in to the last step for our gradient boosting model built from scratch. Classification is very similar to the regression process but differs mainly in the scoring. The reader is referred to Murphy (607) for further reading on classification. There is also a modified gradient boosting algorithm, which improves on what we've covered above called Extreme Gradient Boosting or XGBoost. One of the modifications is that XGBoost uses a 2nd order approximation of the loss function and adds a regularizer on the tree complexity. Refer to Murphy (615) for details on this.

2. Ensemble Learning Comparison

This section will apply Bagging, Stacking, AdaBoost, and Gradient boosting to a common classification problem. We predict whether the Luxembourg index (LUXXX) will exceed a return of 0.25% in any direction. The predictors are a combination of country indices and technical indicators. We start by importing the data.

In [13]: **import** warnings

```
warnings.filterwarnings("ignore")

# loc = "ENTER YOUR FULL PATH TO LOCATION OF DATA FILE HERE"
# data_df = pd.read_csv(loc+"/MScFE 650 MLF GWP Data.csv")

data_df = pd.read_csv("MScFE 650 MLF GWP Data.csv")
# Convert string to datetime
data_df["Date"] = pd.to_datetime(data_df["Date"])
```

Create the target variable.

In [14]: *# Set Target Index for predicting*

```
target ETF = "LUXXX"

# Use returns instead of prices for other Indices
# Other Indices used as Index_features
ETF_features = data_df.loc[:, ~data_df.columns.isin(["Date",
target ETF])]
data_df[ETF_features] = data_df[ETF_features].pct_change()

data_df[target ETF + "_returns"] = data_df[target ETF].pct_change()

# Create Target Column.
# Shift period for target column
data_df[target ETF + "_returns" + "_Shift"] = data_df[target ETF +
"_returns"].shift(-1)

# Strategy to take long position for anticipated returns of 0.5%
data_df["Target"] = np.where(
    (data_df[target ETF + "_returns_Shift"].abs() > 0.025), 1, 0
)
```

Technical indicators included as predictors are slow to fast moving average ratio (SMA_ratio), Relative Strength Index (RSI), and Rate of Change (RC).

```

In [15]: # Four country indices used.
feats = ["MSCI KOREA", "MSCI DENMARK", "MSCI FRANCE", "MSCI NORWAY"]

# creating the technical indicators
data_df["SMA_5"] = data_df[target ETF].rolling(5).mean()
data_df["SMA_15"] = data_df[target ETF].rolling(15).mean()
data_df["SMA_ratio"] = data_df["SMA_15"] / data_df["SMA_5"]

# Can drop SMA columns since not needed anymore.
data_df.drop(["SMA_5", "SMA_15"], axis=1, inplace=True)

# shift the price of the target by 1 unit previous in time
data_df["Diff"] = data_df[target ETF] - data_df[target ETF].shift(1)
data_df["Up"] = data_df["Diff"]
data_df.loc[(data_df["Up"] < 0), "Up"] = 0

data_df["Down"] = data_df["Diff"]
data_df.loc[(data_df["Down"] > 0), "Down"] = 0
data_df["Down"] = abs(data_df["Down"])

data_df["avg_5up"] = data_df["Up"].rolling(5).mean()
data_df["avg_5down"] = data_df["Down"].rolling(5).mean()

data_df["avg_15up"] = data_df["Up"].rolling(15).mean()
data_df["avg_15down"] = data_df["Down"].rolling(15).mean()

data_df["RS_5"] = data_df["avg_5up"] / data_df["avg_5down"]
data_df["RS_15"] = data_df["avg_15up"] / data_df["avg_15down"]

data_df["RSI_5"] = 100 - (100 / (1 + data_df["RS_5"]))
data_df["RSI_15"] = 100 - (100 / (1 + data_df["RS_15"]))

data_df["RSI_ratio"] = data_df["RSI_5"] / data_df["RSI_15"]

# Can drop RS Calc columns columns
data_df.drop(
    ["Diff", "Up", "Down", "avg_5up", "avg_5down", "avg_15up",
    "avg_15down"],
    axis=1,
    inplace=True,
)

data_df["RC"] = data_df[target ETF].pct_change(15)

# all_feats
feats.append("SMA_ratio")
feats.append("RSI_ratio")
feats.append("RC")

```

Now we can apply all our models to the data. Perform the train/test split of 80/20.

```
In [16]: # Train/Test split
# Train/Test split. No NaNs in the data.
NoNaN_df = data_df.dropna()
X = NoNaN_df[feats]

X = X.iloc[:, :] # .values
y = NoNaN_df.loc[:, "Target"] # .values

del NoNaN_df

# from sklearn.cross_validation import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=0)
```

We will tune the boosting models and use the hyperparameters from Lesson 2 for the bagging model. We should note that AdaBoost by default has Decision Tree Classifiers as base learners with `max_depth = 1` as default. For the gradient boosting classifier, we assume the default `max_depth` of 3. We only tune on learning rate and `n_estimators` to reduce computational time. We also do the same for the XGBoost model.

```
In [17]: from sklearn.model_selection import GridSearchCV
```

```
In [18]: # parameters for AdaBoost
param_grid = {"n_estimators": [10, 20, 50, 100], "learning_rate": [0.1,
0.25, 0.5, 1.0]}

gridAdBoost = GridSearchCV(
    AdaBoostClassifier(), param_grid, refit=True, verbose=3, cv=3
)
# fitting the model for grid search
gridAdBoost.fit(X_train, y_train)
```

Fitting 3 folds for each of 16 candidates, totalling 48 fits

[CV 1/3] END learning_rate=0.1, n_estimators=10;; score=0.698 total time=0.0s

[CV 2/3] END learning_rate=0.1, n_estimators=10;; score=0.683 total time=0.0s

[CV 3/3] END learning_rate=0.1, n_estimators=10;; score=0.710 total time=0.0s

[CV 1/3] END learning_rate=0.1, n_estimators=20;; score=0.698 total time=0.0s

[CV 2/3] END learning_rate=0.1, n_estimators=20;; score=0.603 total time=0.0s

[CV 3/3] END learning_rate=0.1, n_estimators=20;; score=0.694 total time=0.0s

[CV 1/3] END learning_rate=0.1, n_estimators=50;; score=0.698 total time=0.0s

[CV 2/3] END learning_rate=0.1, n_estimators=50;; score=0.571 total time=0.0s

[CV 3/3] END learning_rate=0.1, n_estimators=50;; score=0.677 total time=0.0s

[CV 1/3] END learning_rate=0.1, n_estimators=100;; score=0.667 total time=0.1s

[CV 2/3] END learning_rate=0.1, n_estimators=100;; score=0.571 total time=0.1s

[CV 3/3] END learning_rate=0.1, n_estimators=100;; score=0.661 total time=0.1s

[CV 1/3] END learning_rate=0.25, n_estimators=10;; score=0.698 total time=0.0s

[CV 2/3] END learning_rate=0.25, n_estimators=10;; score=0.587 total time=0.0s

[CV 3/3] END learning_rate=0.25, n_estimators=10;; score=0.694 total time=0.0s

[CV 1/3] END learning_rate=0.25, n_estimators=20;; score=0.683 total time=0.0s

[CV 2/3] END learning_rate=0.25, n_estimators=20;; score=0.571 total time=0.0s

[CV 3/3] END learning_rate=0.25, n_estimators=20;; score=0.677 total time=0.0s

[CV 1/3] END learning_rate=0.25, n_estimators=50;; score=0.635 total time=0.0s

[CV 2/3] END learning_rate=0.25, n_estimators=50;; score=0.603 total time=0.1s

[CV 3/3] END learning_rate=0.25, n_estimators=50;; score=0.645 total time=0.1s

[CV 1/3] END learning_rate=0.25, n_estimators=100;; score=0.651 total time=0.1s

[CV 2/3] END learning_rate=0.25, n_estimators=100;; score=0.540 total time=0.1s

[CV 3/3] END learning_rate=0.25, n_estimators=100;; score=0.677 total time=0.1s

[CV 1/3] END learning_rate=0.5, n_estimators=10;; score=0.683 total time=0.0s

[CV 2/3] END learning_rate=0.5, n_estimators=10;; score=0.635 total time=0.0s

[CV 3/3] END learning_rate=0.5, n_estimators=10;; score=0.629 total time=0.0s

[CV 1/3] END learning_rate=0.5, n_estimators=20;; score=0.619 total time=0.0s

[CV 2/3] END learning_rate=0.5, n_estimators=20;; score=0.603 total time=0.0s

[CV 3/3] END learning_rate=0.5, n_estimators=20;; score=0.645 total time=


```
In [20]: # parameters for Gradient Boosting Classifier
param_grid = {"n_estimators": [10, 20, 50, 100], "learning_rate": [0.1,
0.25, 0.5, 1.0]}

GBgrid = GridSearchCV(
    GradientBoostingClassifier(), param_grid, refit=True, verbose=3, cv=3
)

# fitting the model for grid search
GBgrid.fit(X_train, y_train)
```

Fitting 3 folds for each of 16 candidates, totalling 48 fits

[CV 1/3] END learning_rate=0.1, n_estimators=10;; score=0.683 total time=0.0s

[CV 2/3] END learning_rate=0.1, n_estimators=10;; score=0.619 total time=0.0s

[CV 3/3] END learning_rate=0.1, n_estimators=10;; score=0.645 total time=0.0s

[CV 1/3] END learning_rate=0.1, n_estimators=20;; score=0.667 total time=0.0s

[CV 2/3] END learning_rate=0.1, n_estimators=20;; score=0.619 total time=0.0s

[CV 3/3] END learning_rate=0.1, n_estimators=20;; score=0.661 total time=0.0s

[CV 1/3] END learning_rate=0.1, n_estimators=50;; score=0.667 total time=0.0s

[CV 2/3] END learning_rate=0.1, n_estimators=50;; score=0.619 total time=0.0s

[CV 3/3] END learning_rate=0.1, n_estimators=50;; score=0.677 total time=0.0s

[CV 1/3] END learning_rate=0.1, n_estimators=100;; score=0.619 total time=0.1s

[CV 2/3] END learning_rate=0.1, n_estimators=100;; score=0.635 total time=0.1s

[CV 3/3] END learning_rate=0.1, n_estimators=100;; score=0.694 total time=0.1s

[CV 1/3] END learning_rate=0.25, n_estimators=10;; score=0.635 total time=0.0s

[CV 2/3] END learning_rate=0.25, n_estimators=10;; score=0.619 total time=0.0s

[CV 3/3] END learning_rate=0.25, n_estimators=10;; score=0.645 total time=0.0s

[CV 1/3] END learning_rate=0.25, n_estimators=20;; score=0.603 total time=0.0s

[CV 2/3] END learning_rate=0.25, n_estimators=20;; score=0.635 total time=0.0s

[CV 3/3] END learning_rate=0.25, n_estimators=20;; score=0.661 total time=0.0s

[CV 1/3] END learning_rate=0.25, n_estimators=50;; score=0.556 total time=0.0s

[CV 2/3] END learning_rate=0.25, n_estimators=50;; score=0.651 total time=0.0s

[CV 3/3] END learning_rate=0.25, n_estimators=50;; score=0.694 total time=0.0s

[CV 1/3] END learning_rate=0.25, n_estimators=100;; score=0.603 total time=0.1s

[CV 2/3] END learning_rate=0.25, n_estimators=100;; score=0.587 total time=0.1s

[CV 3/3] END learning_rate=0.25, n_estimators=100;; score=0.645 total time=0.1s

[CV 1/3] END learning_rate=0.5, n_estimators=10;; score=0.587 total time=0.0s

[CV 2/3] END learning_rate=0.5, n_estimators=10;; score=0.587 total time=0.0s

[CV 3/3] END learning_rate=0.5, n_estimators=10;; score=0.645 total time=0.0s

[CV 1/3] END learning_rate=0.5, n_estimators=20;; score=0.603 total time=0.0s

[CV 2/3] END learning_rate=0.5, n_estimators=20;; score=0.619 total time=0.0s

[CV 3/3] END learning_rate=0.5, n_estimators=20;; score=0.629 total time=


```
In [23]: # parameters for XG Boosting Classifier
param_grid = {"n_estimators": [10, 20, 50, 100], "learning_rate": [0.1,
0.25, 0.5, 1.0]}

XGB_model = xgb.XGBClassifier()
XGBgrid = GridSearchCV(XGB_model, param_grid, refit=True, verbose=3, cv=3)

# fitting the model for grid search
XGBgrid.fit(X_train, y_train)
```

Fitting 3 folds for each of 16 candidates, totalling 48 fits

[CV 1/3] END learning_rate=0.1, n_estimators=10;; score=0.603 total time=0.0s

[CV 2/3] END learning_rate=0.1, n_estimators=10;; score=0.683 total time=0.0s

[CV 3/3] END learning_rate=0.1, n_estimators=10;; score=0.677 total time=0.0s

[CV 1/3] END learning_rate=0.1, n_estimators=20;; score=0.635 total time=0.0s

[CV 2/3] END learning_rate=0.1, n_estimators=20;; score=0.698 total time=0.0s

[CV 3/3] END learning_rate=0.1, n_estimators=20;; score=0.661 total time=0.0s

[CV 1/3] END learning_rate=0.1, n_estimators=50;; score=0.651 total time=0.0s

[CV 2/3] END learning_rate=0.1, n_estimators=50;; score=0.683 total time=0.0s

[CV 3/3] END learning_rate=0.1, n_estimators=50;; score=0.661 total time=0.0s

[CV 1/3] END learning_rate=0.1, n_estimators=100;; score=0.635 total time=0.0s

[CV 2/3] END learning_rate=0.1, n_estimators=100;; score=0.619 total time=0.0s

[CV 3/3] END learning_rate=0.1, n_estimators=100;; score=0.661 total time=0.0s

[CV 1/3] END learning_rate=0.25, n_estimators=10;; score=0.635 total time=0.0s

[CV 2/3] END learning_rate=0.25, n_estimators=10;; score=0.683 total time=0.0s

[CV 3/3] END learning_rate=0.25, n_estimators=10;; score=0.677 total time=0.0s

[CV 1/3] END learning_rate=0.25, n_estimators=20;; score=0.635 total time=0.0s

[CV 2/3] END learning_rate=0.25, n_estimators=20;; score=0.667 total time=0.0s

[CV 3/3] END learning_rate=0.25, n_estimators=20;; score=0.661 total time=0.0s

[CV 1/3] END learning_rate=0.25, n_estimators=50;; score=0.635 total time=0.0s

[CV 2/3] END learning_rate=0.25, n_estimators=50;; score=0.635 total time=0.0s

[CV 3/3] END learning_rate=0.25, n_estimators=50;; score=0.629 total time=0.0s

[CV 1/3] END learning_rate=0.25, n_estimators=100;; score=0.651 total time=0.0s

[CV 2/3] END learning_rate=0.25, n_estimators=100;; score=0.635 total time=0.0s

[CV 3/3] END learning_rate=0.25, n_estimators=100;; score=0.629 total time=0.0s

[CV 1/3] END learning_rate=0.5, n_estimators=10;; score=0.651 total time=0.0s

[CV 2/3] END learning_rate=0.5, n_estimators=10;; score=0.619 total time=0.0s

[CV 3/3] END learning_rate=0.5, n_estimators=10;; score=0.661 total time=0.0s

[CV 1/3] END learning_rate=0.5, n_estimators=20;; score=0.667 total time=0.0s

[CV 2/3] END learning_rate=0.5, n_estimators=20;; score=0.651 total time=0.0s

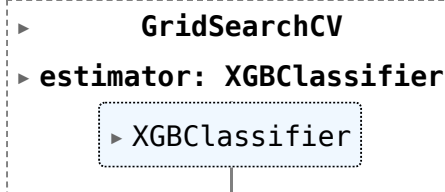
[CV 3/3] END learning_rate=0.5, n_estimators=20;; score=0.661 total time=0.0s

```

0.0s
[CV 1/3] END learning_rate=0.5, n_estimators=50;, score=0.667 total time=
0.0s
[CV 2/3] END learning_rate=0.5, n_estimators=50;, score=0.571 total time=
0.0s
[CV 3/3] END learning_rate=0.5, n_estimators=50;, score=0.645 total time=
0.0s
[CV 1/3] END learning_rate=0.5, n_estimators=100;, score=0.651 total time=
0.0s
[CV 2/3] END learning_rate=0.5, n_estimators=100;, score=0.571 total time=
0.0s
[CV 3/3] END learning_rate=0.5, n_estimators=100;, score=0.613 total time=
0.0s
[CV 1/3] END learning_rate=1.0, n_estimators=10;, score=0.603 total time=
0.0s
[CV 2/3] END learning_rate=1.0, n_estimators=10;, score=0.635 total time=
0.0s
[CV 3/3] END learning_rate=1.0, n_estimators=10;, score=0.597 total time=
0.0s
[CV 1/3] END learning_rate=1.0, n_estimators=20;, score=0.667 total time=
0.0s
[CV 2/3] END learning_rate=1.0, n_estimators=20;, score=0.619 total time=
0.0s
[CV 3/3] END learning_rate=1.0, n_estimators=20;, score=0.629 total time=
0.0s
[CV 1/3] END learning_rate=1.0, n_estimators=50;, score=0.619 total time=
0.0s
[CV 2/3] END learning_rate=1.0, n_estimators=50;, score=0.587 total time=
0.0s
[CV 3/3] END learning_rate=1.0, n_estimators=50;, score=0.613 total time=
0.0s
[CV 1/3] END learning_rate=1.0, n_estimators=100;, score=0.619 total time=
0.0s
[CV 2/3] END learning_rate=1.0, n_estimators=100;, score=0.556 total time=
0.0s
[CV 3/3] END learning_rate=1.0, n_estimators=100;, score=0.645 total time=
0.0s

```

Out[23]:



In [24]: *# print best parameter after tuning*
print(XGBgrid.best_params_)

print how our XGBoost model looks after hyper-parameter tuning
print(XGBgrid.best_estimator_)

```
{'learning_rate': 0.25, 'n_estimators': 10}
XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bytree=None, device=None,
early_stopping_rounds=None,
               enable_categorical=False, eval_metric=None,
feature_types=None,
               feature_weights=None, gamma=None, grow_policy=None,
               importance_type=None, interaction_constraints=None,
               learning_rate=0.25, max_bin=None, max_cat_threshold=None,
               max_cat_to_onehot=None, max_delta_step=None, max_depth=None,
               max_leaves=None, min_child_weight=None, missing=nan,
               monotone_constraints=None, multi_strategy=None,
n_estimators=10,
               n_jobs=None, num_parallel_tree=None, ...)
```

So we find that AdaBoost and Gradient Boosting have the same optimal hyperparameters; however, XGBoost has a different learning rate. We can train all of our ensemble models and then compare performance afterwards.

```
In [25]: # Train with Tuned Random Forest
# Create a tuned RF Classifier
bagmodel_tuned = RandomForestClassifier(
    max_depth=2, min_samples_split=8, n_estimators=10, random_state=10
)

bagmodel_tuned.fit(X_train, y_train)
```

```
Out[25]: ▼ RandomForestClassifier
RandomForestClassifier(max_depth=2, min_samples_split=8,
n_estimators=10,
                    random_state=10)
```

This is the stacking model.

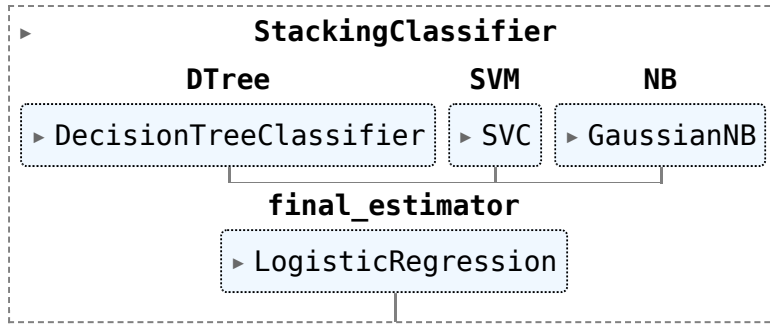
```
In [26]: clf1 = DecisionTreeClassifier() # Decision Tree

clf2 = SVC(kernel="rbf") # Support Vector Classifier

clf3 = GaussianNB() # Gaussian Naive Bayes

est_rs = [("DTree", clf1), ("SVM", clf2), ("NB", clf3)]
# Meta model
mylr = LogisticRegression()
# creating a stacking classifier
stackingCLF = StackingClassifier(
    estimators=est_rs, final_estimator=mylr, stack_method="auto", cv=3
)
stackingCLF.fit(X_train, y_train)
```

Out[26]:



Fit all tuned boosting models to the training set.

```
In [27]: # Create a tuned AdaBoost Classifier
AdaBoost_tuned = AdaBoostClassifier(learning_rate=0.1, n_estimators=10)

# Create a tuned Gradient Boosting Classifier
GB_tuned = GradientBoostingClassifier(learning_rate=0.1, n_estimators=10)

# Create a tuned XGBoost Classifier
XGB_tuned = xgb.XGBClassifier(learning_rate=0.25, n_estimators=10)

# train boosting models
AdaBoost_tuned.fit(X_train, y_train)
GB_tuned.fit(X_train, y_train)
XGB_tuned.fit(X_train, y_train)
print("Training complete")
```

Training complete

Lastly, we compare all models with an ROC curve.

```
In [28]: from sklearn.metrics import roc_auc_score, roc_curve
```

```

In [29]: # predicted probabilities generated by models
y_pred_probaStack = stackingCLF.predict_proba(X_test) # stacking
y_pred_probaRF = bagmodel_tuned.predict_proba(X_test) # RF
y_pred_probaAdB = AdaBoost_tuned.predict_proba(X_test) # AdaBoost
y_pred_probaGb = GB_tuned.predict_proba(X_test) # `GradBoost`
y_pred_probaXGB = XGB_tuned.predict_proba(X_test) # XGBoost

# Stacking ROC dependencies
fpr, tpr, _ = roc_curve(y_test, y_pred_probaStack[:, 1])
auc = round(roc_auc_score(y_test, y_pred_probaStack[:, 1]), 4)

# RF ROC dependencies
fpr_RF, tpr_RF, _ = roc_curve(y_test, y_pred_probaRF[:, 1])
auc_RF = round(roc_auc_score(y_test, y_pred_probaRF[:, 1]), 4)

# AdaBoost ROC dependencies
fpr_AB, tpr_AB, _ = roc_curve(y_test, y_pred_probaAdB[:, 1])
auc_AB = round(roc_auc_score(y_test, y_pred_probaAdB[:, 1]), 4)

# Gradient Boost ROC dependencies
fpr_GB, tpr_GB, _ = roc_curve(y_test, y_pred_probaGb[:, 1])
auc_GB = round(roc_auc_score(y_test, y_pred_probaGb[:, 1]), 4)

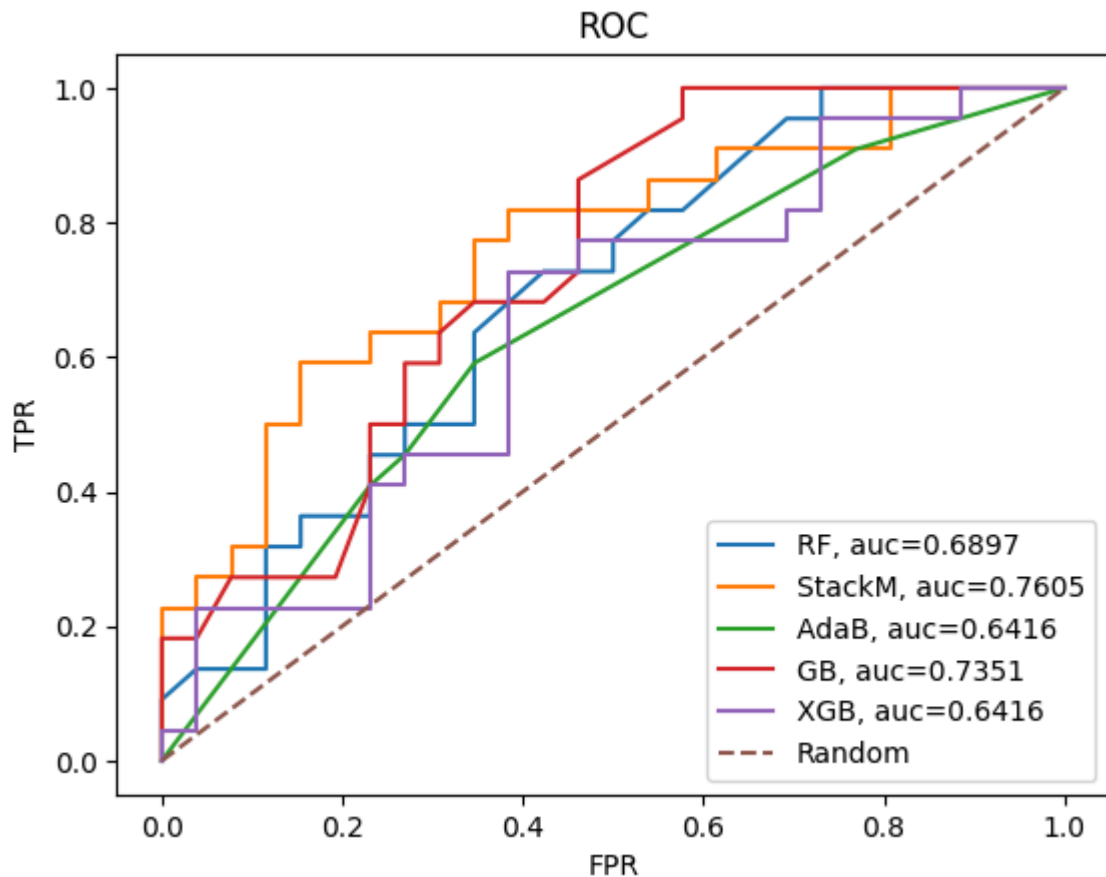
# XGB ROC dependencies
fpr_XGB, tpr_XGB, _ = roc_curve(y_test, y_pred_probaXGB[:, 1])
auc_XGB = round(roc_auc_score(y_test, y_pred_probaXGB[:, 1]), 4)

# RF Model
plt.plot(fpr_RF, tpr_RF, label="RF, auc=" + str(auc_RF))
# Stacking Model
plt.plot(fpr, tpr, label="StackM, auc=" + str(auc))
# AdaBoost Model
plt.plot(fpr_AB, tpr_AB, label="AdaB, auc=" + str(auc_AB))
# `GradBoost` Model
plt.plot(fpr_GB, tpr_GB, label="GB, auc=" + str(auc_GB))
# XGBoost Model
plt.plot(fpr_XGB, tpr_XGB, label="XGB, auc=" + str(auc_XGB))

# Random guess model
plt.plot(fpr, fpr, "--", label="Random")
plt.title("ROC")
plt.ylabel("TPR")
plt.xlabel("FPR")

plt.legend(loc=4)
plt.show()

```



Great! All models performed reasonably well with some models exceeding 70% AUC. Keep in mind that we did not explore all hyperparameters for models, but this is a good enough starting point to create your own models and explore the hyperparameter space and effect on model performance.

3. Conclusion

This lesson explored the last ensemble learning algorithm, namely gradient boosting in more detail after briefly mentioning it in Module 3. We compared all ensemble learning algorithms covered in this module in a common classification problem and obtained results significantly better than a no-skill model, thus showing the added value in using the combined predictive power of weak learners.

References

1. Murphy, Kevin P. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
2. University of California, Irvine Machine Learning Repository. "Real Estate Valuation Data Set."
<https://archive.ics.uci.edu/ml/datasets/Real+estate+valuation+data+set>

